Volcano for the Agentic AI Era:

Unified Scheduling Across Training, Inference, and Agents

Ricardo Aravena · Rainie Li

KubeCon + CloudNativeCon North America 2026
Salt Lake City · November 9–12

Speakers

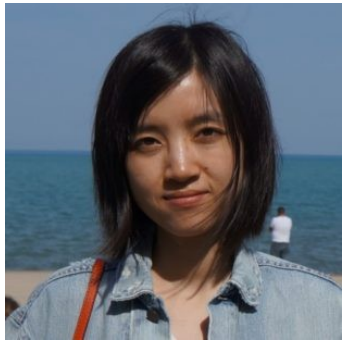


Ricardo Aravena

Cloud Native & AI Infrastructure

Open-source infrastructure
Kubernetes and AI/ML platforms

serverbooter.com



Rainie Li

Pinterest

Data processing infrastructure
Apache YuniKorn PMC member

Apache YuniKorn · rainieli

01

Unified scheduling

01 Unified scheduling

02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

Different paths. Shared infrastructure.

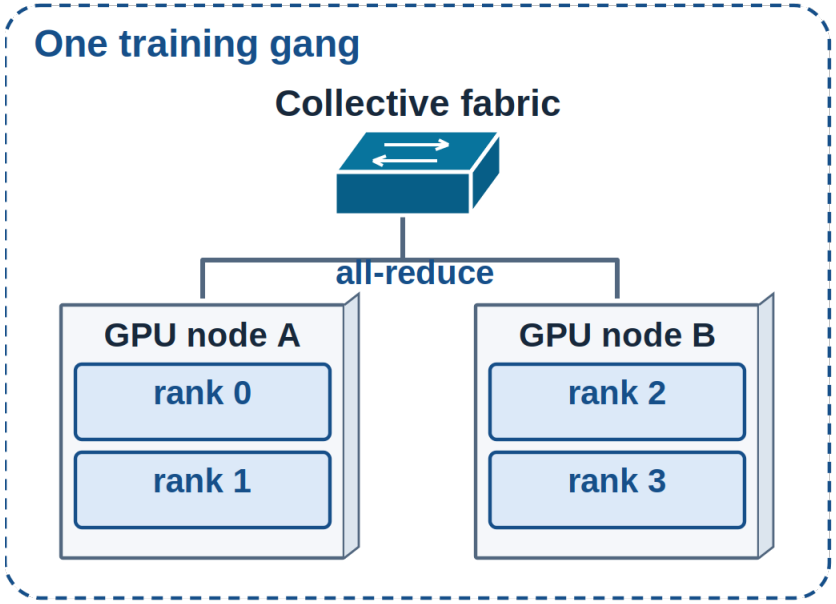
How do schedulers coordinate node access?

Different Workloads, Shared Infrastructure

Training, inference, and agent applications put different demands on the same cluster.

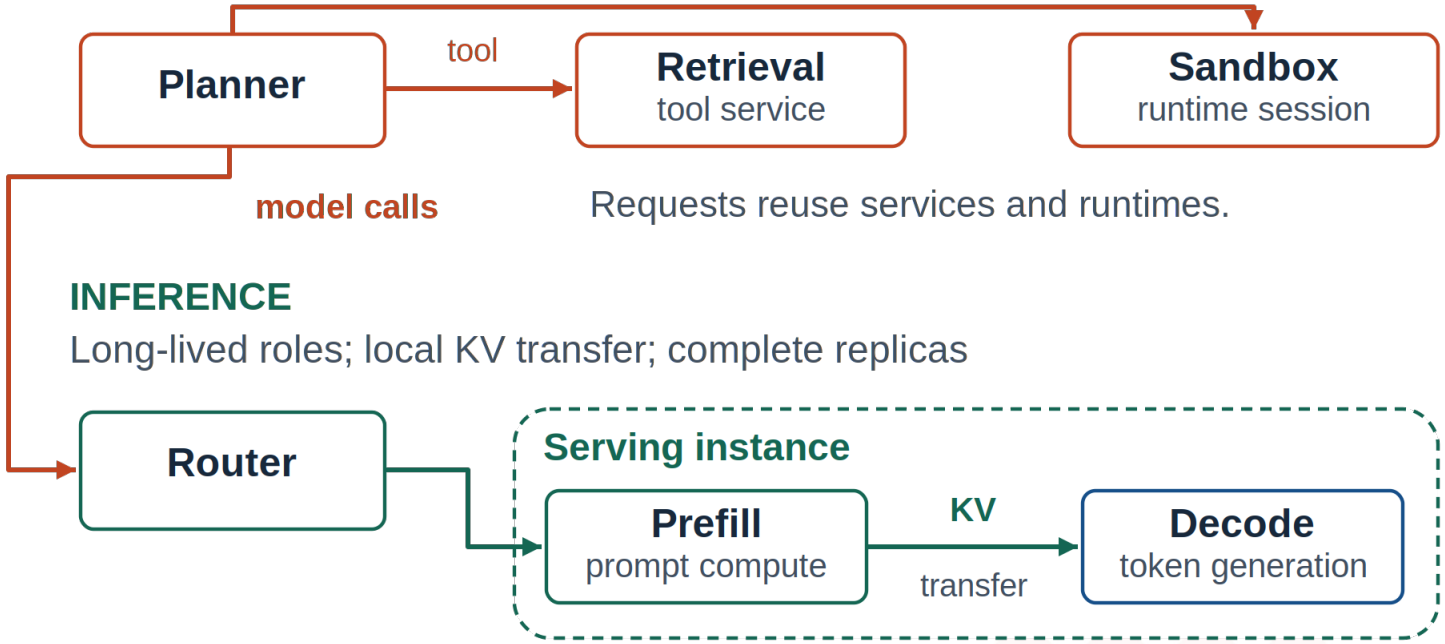
TRAINING & RL

Complete compute groups; collective communication



AGENT APPLICATION

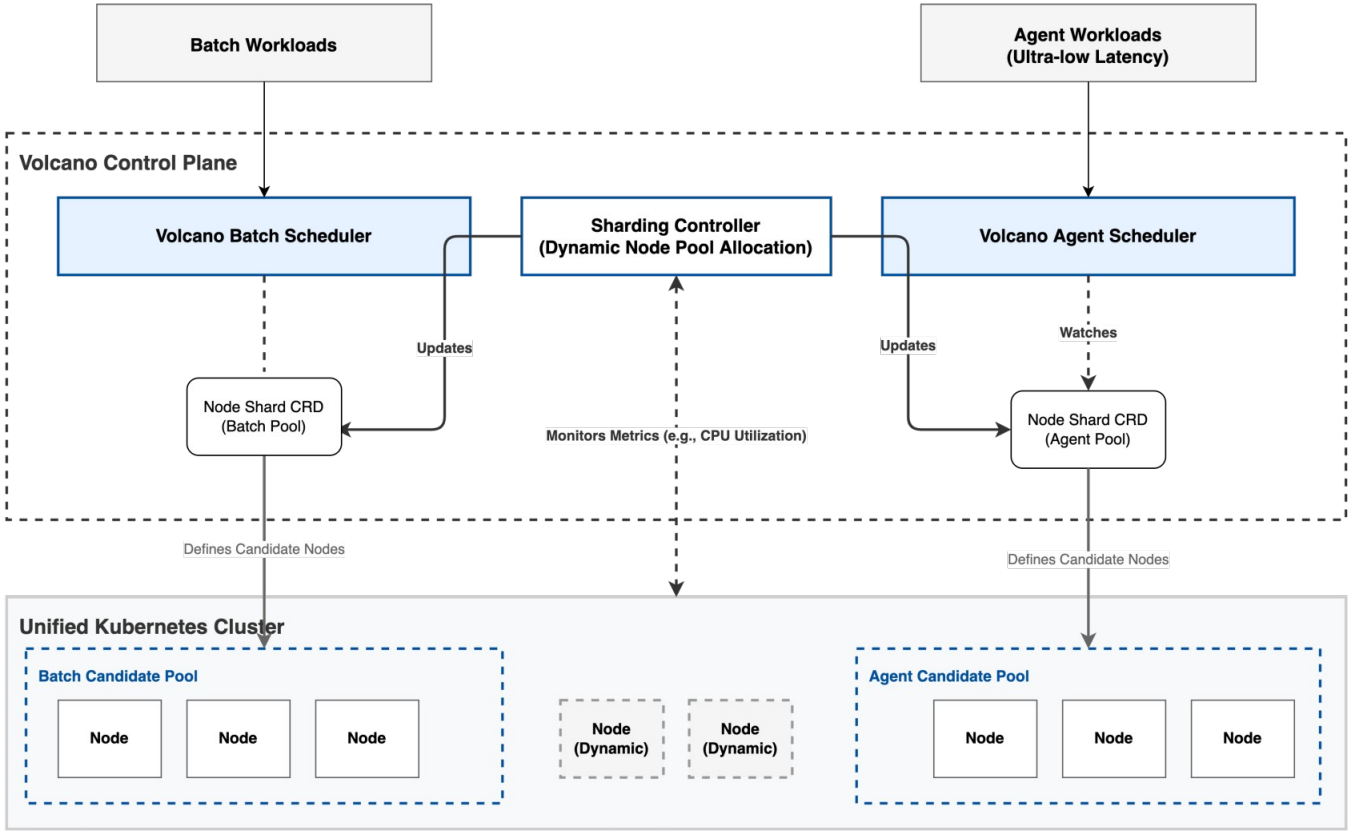
Variable fan-out; latency-sensitive runtime calls



Shared Kubernetes capacity · compute groups, serving replicas, and runtime demand are different scheduling concerns.

A Coordinated Scheduling Framework

Specialized scheduling paths coordinate through NodeShard.



01 Workload paths

Batch-oriented scheduling and a Pod-oriented fast path.

02 Local coordination

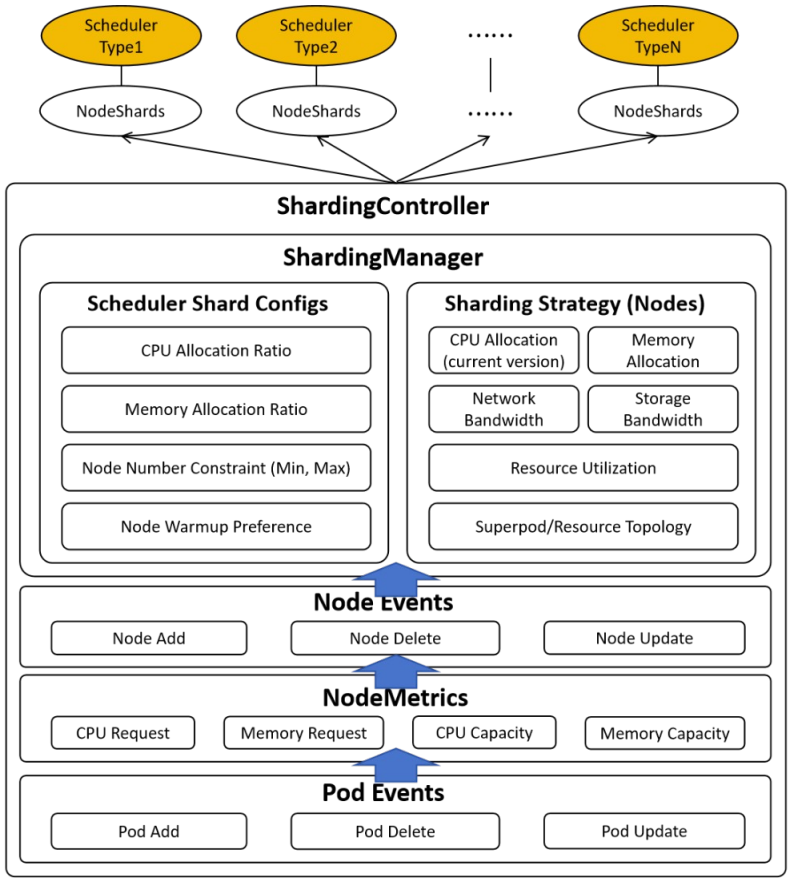
Parallel workers use Binder conflict checks.

03 Shared node access

NodeShard coordinates candidate-node pools.

NodeShard: Desired Nodes and Current Use

Changing a shard coordinates future placement; it does not migrate running Pods.



NodeShard · spec / status excerpt

```
spec:
  nodesDesired:
  - node1
  - node2
status:
  nodesInUse:
  - node0
  - node1
  nodesToRemove:
  - node0
  nodesToAdd:
  - node2
```

Desired membership ≠ immediate use

Coordinators reconcile node access.

02

Multi-level gang & network topology

From available nodes to runnable workloads.

What must run together, and where should it fit?

01 Unified scheduling

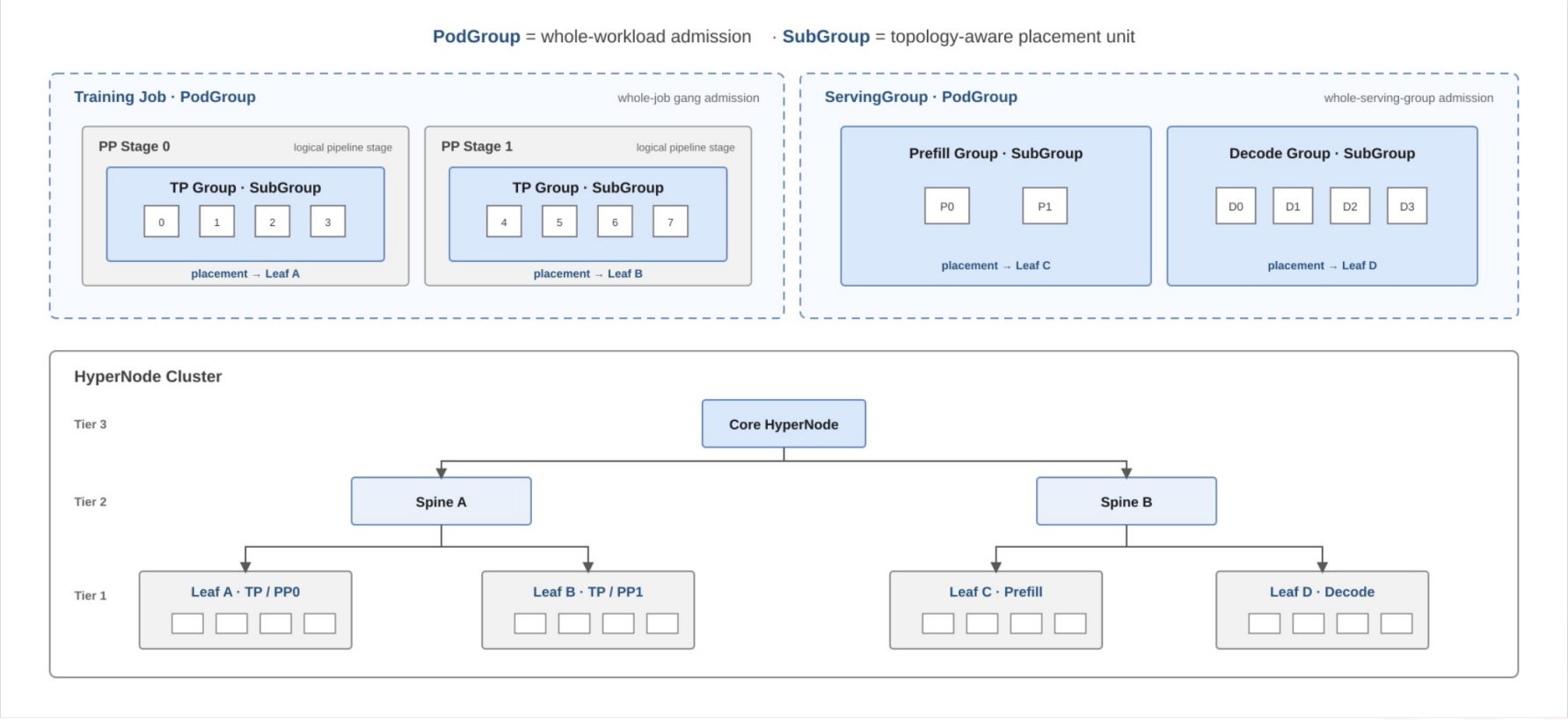
02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

PodGroup and SubGroup: Preserve Workload Structure

Gang requirements apply to the workload and to its complete subgroups.

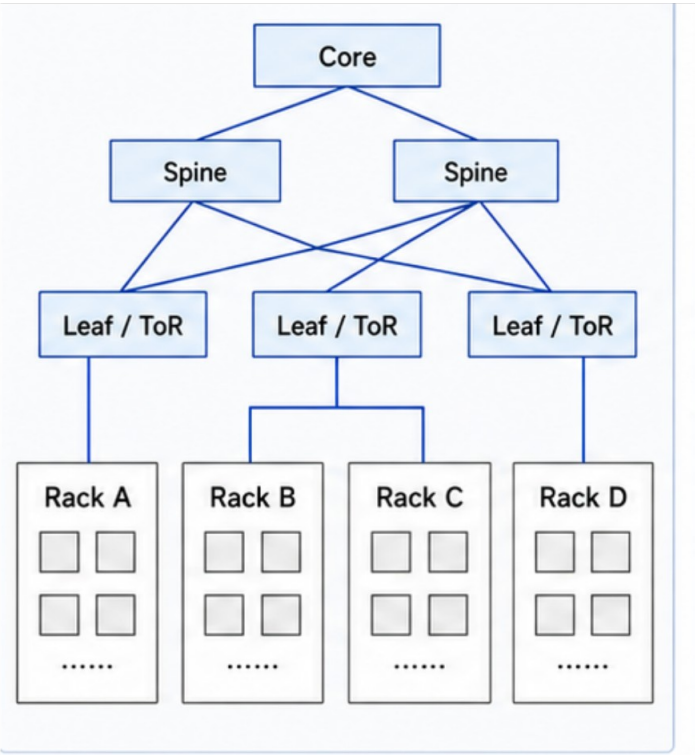


A SubGroup policy selects Pods; replica identity can split them into multiple SubGroups.

HyperNode: From Physical Fabric to Scheduling Domains

A parent HyperNode groups child HyperNodes; a leaf groups Kubernetes Nodes.

PHYSICAL FABRIC



Parent · HyperNode.spec

```
tier: 2
tierName: spine
members:
- type: HyperNode
  selector:
    exactMatch:
      name: rack-a
```

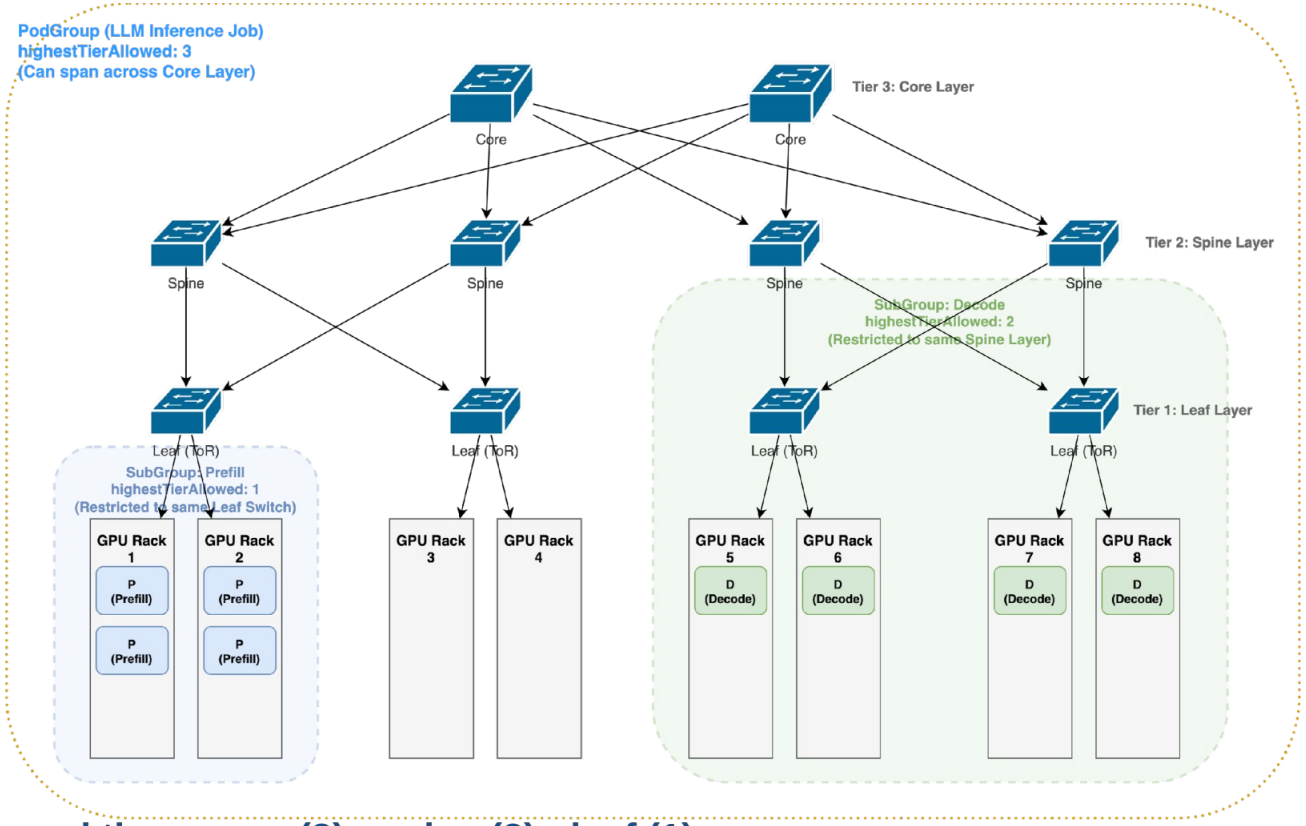
Leaf · HyperNode.spec

```
tier: 1
tierName: leaf
members:
- type: Node
  selector:
    labelMatch:
      matchLabels:
        rack: rack-a
```

The same hierarchy is available through the standalone HyperNode component.

Place Groups at Different Topology Levels

An eight-Pod instance spans a larger domain; Prefill and Decode use different locality constraints.



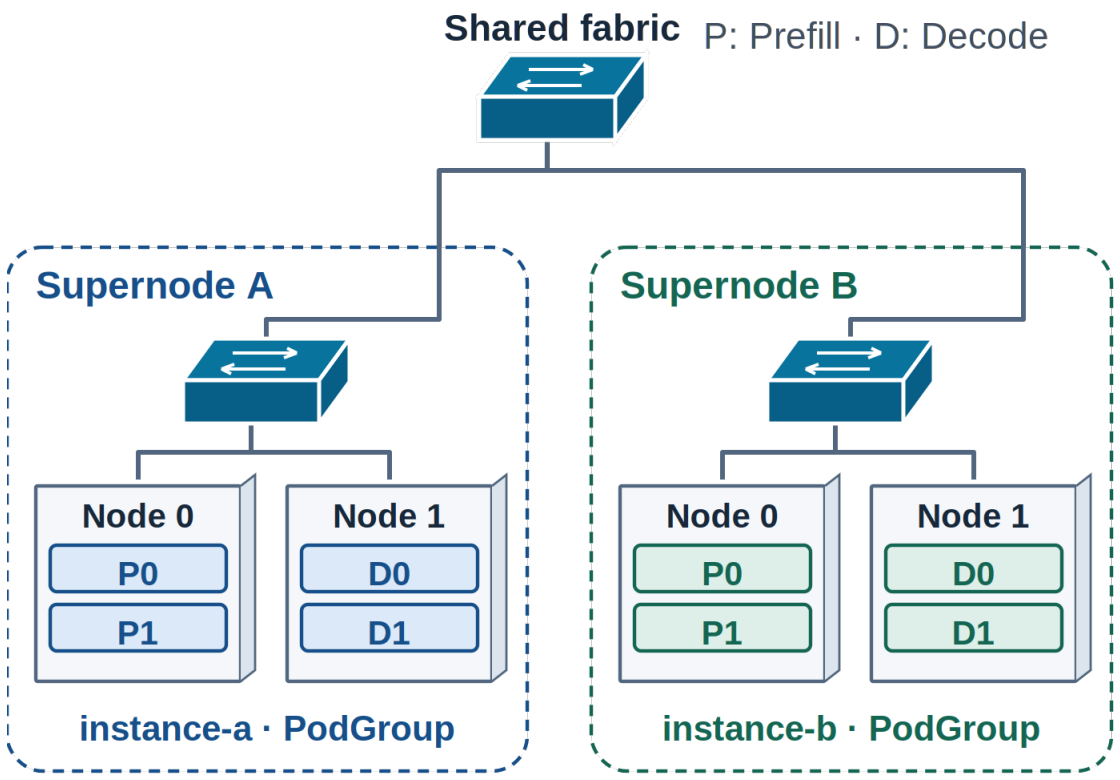
Named tiers: core (3) · spine (2) · leaf (1)

PodGroup.spec · key fields

```
minMember: 8
networkTopology:
  mode: hard
  highestTierName: core
subGroupPolicy:
- name: prefill
  subGroupSize: 4
  networkTopology:
    mode: hard
    highestTierName: leaf
- name: decode
  subGroupSize: 4
  networkTopology:
    mode: hard
    highestTierName: spine
```

Keep Each Instance Together, Place Instances Apart

PodGroup anti-affinity separates service instances across HyperNode domains.



PodGroup · same rule on each instance

```yaml
metadata:
  labels:
    app: model-a
spec:
  networkTopology:
    mode: hard
    highestTierName: supernode
  topologyAffinity:
    podGroupAntiAffinity:
      required:
      - podGroupSelector:
          matchLabels:
            app: model-a
        topologyTierName: supernode
```

Locality keeps each instance within one domain; anti-affinity places matching PodGroups in different domains.

03

Ecosystem

01 Unified scheduling

02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

Workload controllers carry scheduling intent.

How do roles and replicas reach the scheduler?

Controllers Bring Workload Structure to Scheduling

Existing integrations and the next steps in role, replica, and topology support.

INTEGRATIONS & CURRENT GAPS

Kthena

ModelServing → role replicas

Gang and topology available.

KubeRay

Multi-host replica → SubGroup

Gang merged; topology in progress.

Grove

PodClique / scaling group → SubGroup

Backend merged; topology mapping remains.

NEXT COMMUNITY WORK

RBG — IN PROGRESS

RoleInstance → SubGroup foundation.

Continue the integration with the community.

Kubeflow — PLANNED

Build on the existing Volcano plugin.

Extend finer grouping and topology support.

LWS — PLANNED · MULTI-GROUP

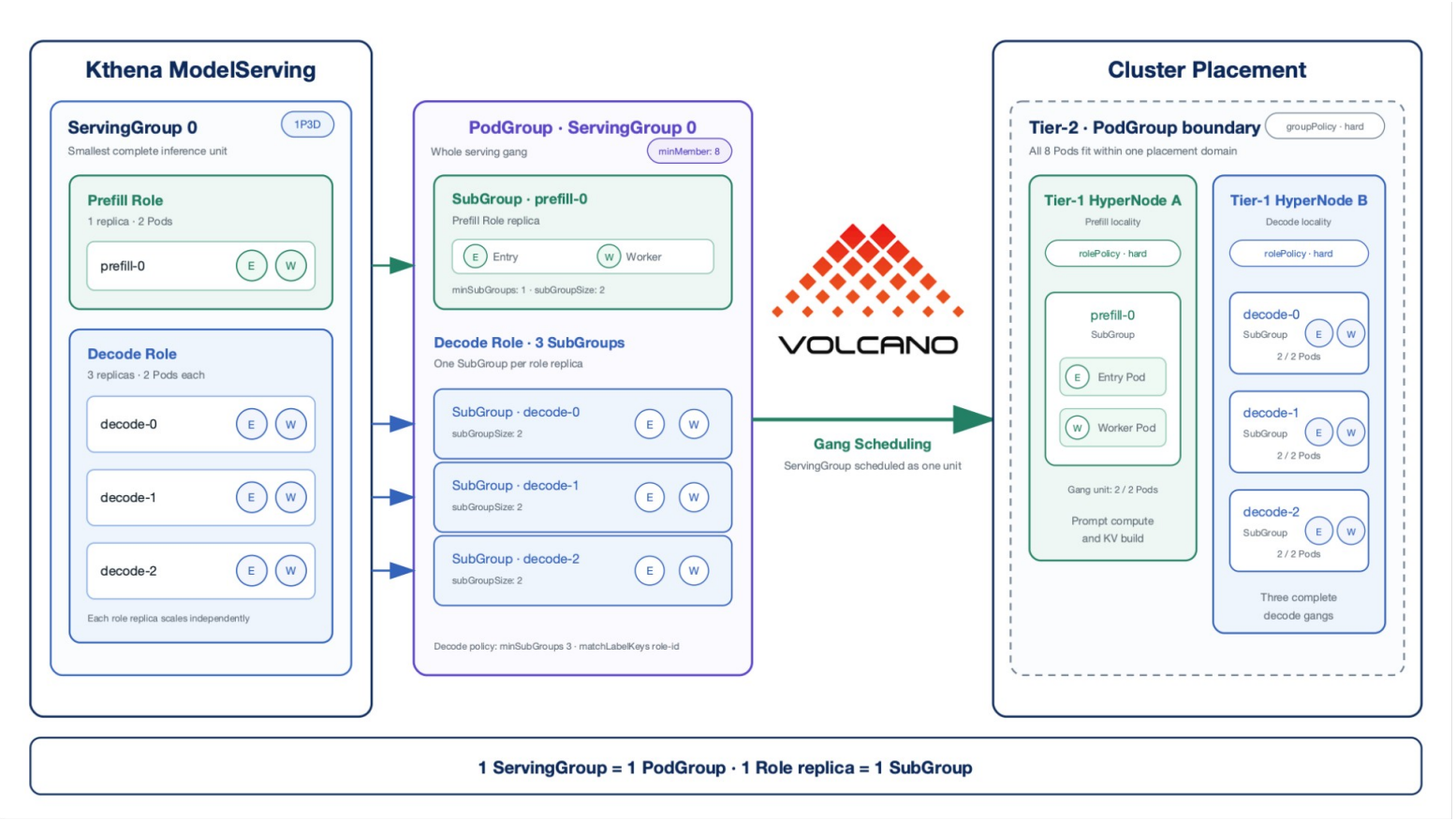
Build on existing gang scheduling.

Connect multi-role groups to scheduling intent.

The next integration step: preserve roles, replica identity, and locality across controller boundaries.

Kthena: From Serving Roles to Scheduling Groups

One ServingGroup maps to one PodGroup; each role replica forms a SubGroup.



ModelServing.spec · excerpt

```
schedulerName: volcano
replicas: 1
template:
  gangPolicy:
    minRoleReplicas:
      prefill: 1
      decode: 3
  roles:
  - name: prefill
    replicas: 1
    workerReplicas: 1
  - name: decode
    replicas: 3
    workerReplicas: 1
```

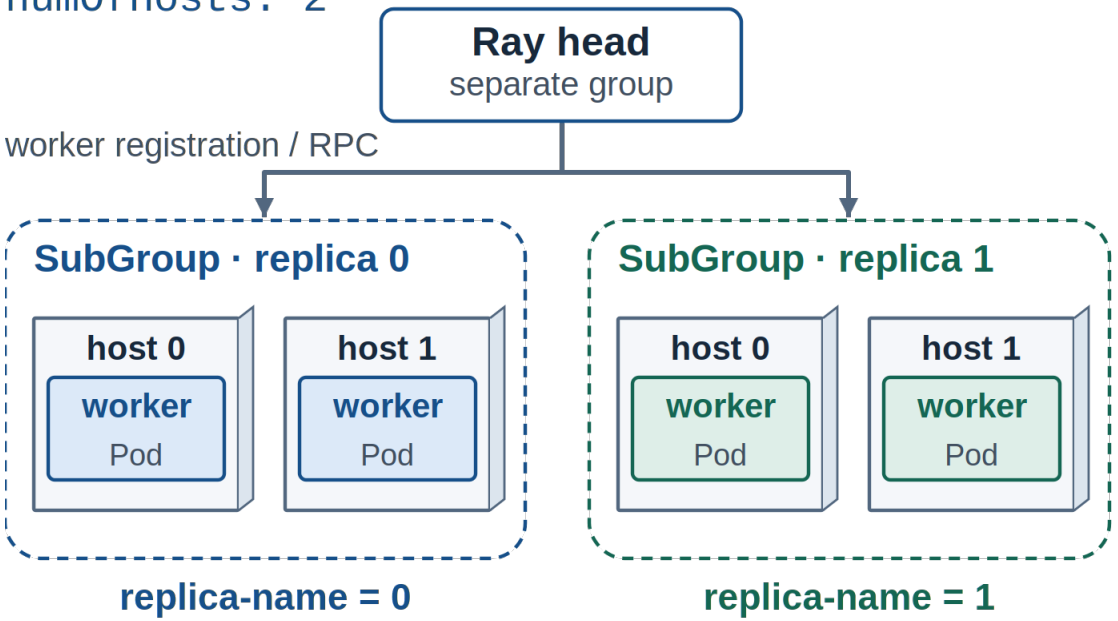
Pod templates omitted.

KubeRay: A Replica Can Span Multiple Pods

The replica label separates complete groups inside one WorkerGroup.

RayCluster · worker group

```
minReplicas: 2
numOfHosts: 2
```



PodGroup.spec · generated grouping

```
subGroupPolicy:
- name: worker
  labelSelector:
    matchLabels:
      ray.io/group: worker
  matchLabelKeys:
  - ray.io/worker-group-replica-name
  minSubGroups: 2
  subGroupSize: 2
```

One replica label → one complete SubGroup

Two replicas × two hosts = four worker Pods

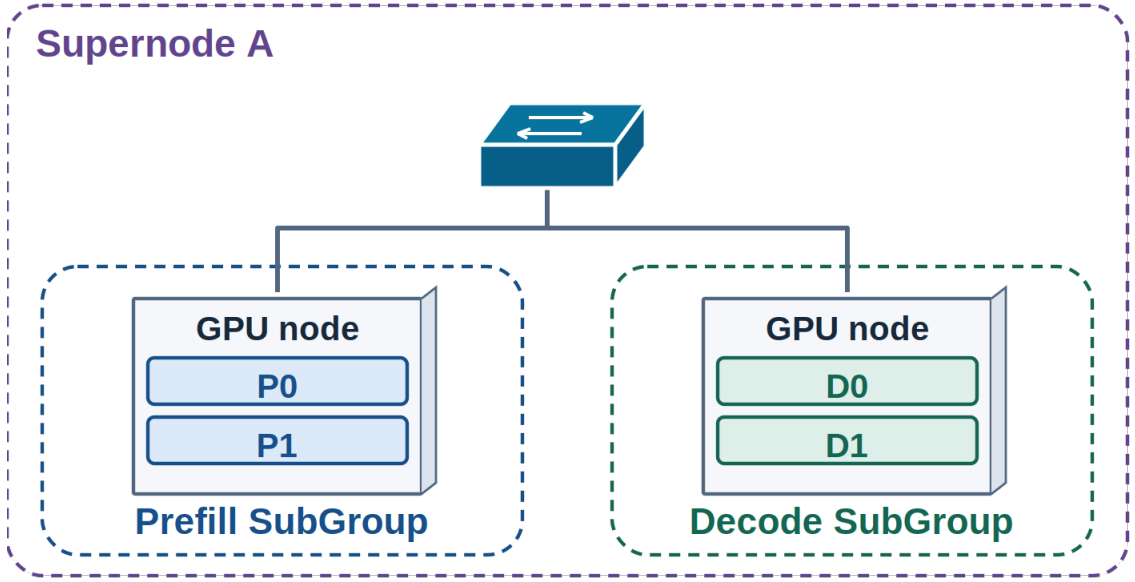
Grouping view, not a topology guarantee. Gang support is merged; replica topology is still in progress.

Next: Relationships Between SubGroups

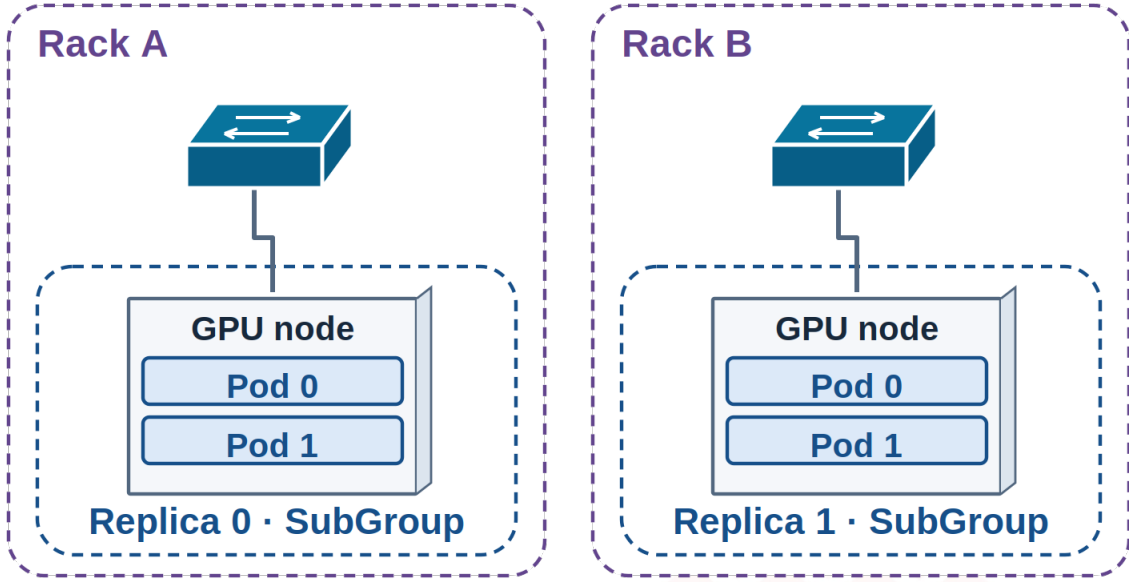
Extend the placement model as controllers expose richer workload structure.

PROPOSED · SubGroup relationships within one PodGroup

Affinity: cooperating groups



Anti-affinity: independent replicas



Place cooperating SubGroups together; spread replicas across domains. These relationships remain proposed.

04

Benchmark framework & progress

Measure behavior across workload shapes.

What can we reproduce, compare, and improve?

01 Unified scheduling

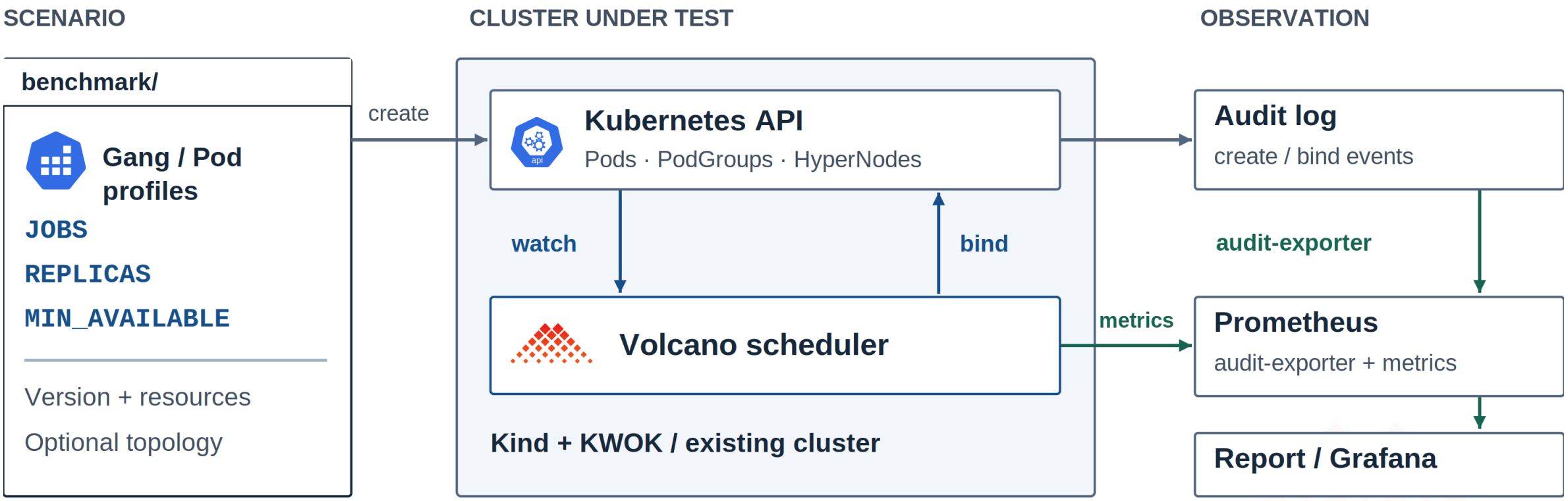
02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

A Repeatable Scheduling Benchmark Framework

Generate scenarios, observe API events, and retain comparable measurements.



The harness controls the scenario and records observations; KWOK does not execute GPU work.

Benchmark Progress and Ongoing Validation

The harness is available; testing and optimization continue across workload scenarios.

RUN A SCENARIO · benchmark/

```
make setup

make test-gang-env \
  JOBS=10 REPLICAS=100 \
  MIN_AVAILABLE=100
```

COLLECT THE EVIDENCE

`logs/kube-apiserver-audit.log`

`results/report-<timestamp>.json`

ONGOING VALIDATION

01 Vary one dimension

Workload size, grouping, or topology.

02 Keep the comparison fixed

Version, resources, and metric definitions.

03 Repeat and inspect

Retain raw events; investigate bottlenecks.

Available: scenario setup, Gang / Pod workloads, topology, metrics, and report collection.

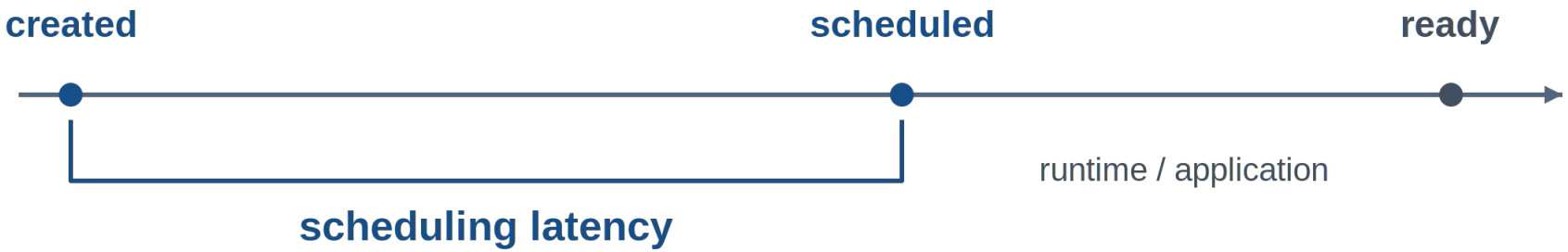
From Measurements to Reliable Comparisons

Keep scheduling latency distinct from application readiness.

MEASUREMENT DEFINITIONS · schematic, not benchmark results

Per-Pod latency

Created → scheduled



Scheduling throughput

Successful binds / window



Compare latency distributions and throughput with the same version, workload, resources, and window.

Framework, Workloads, Ecosystem, Evidence

Four connected pieces of the scheduling story.

01 **Coordinate the paths**

NodeShard + scheduler coordination

02 **Preserve workload structure**

Complete groups + topology + separation

03 **Carry intent through controllers**

Roles and replica identity reach scheduling

04 **Measure and keep improving**

Repeatable scenarios + comparable observations

Questions & Discussion

Workload shapes, integration feedback, and reproducible observations are welcome.

What must run together?

What should stay apart?

Bring a workload, a topology constraint, or a reproducible test.

github.com/volcano-sh/volcano

Roadmap and feedback · issue #6166